

Programación Orientada a Objetos en Python y Java.

Nombre: Josué Miguel Ortega Ludeña

Ciclo: Segundo

Paralelo: D

Docente: Wayner Xavier Bustamante Granda

Periodo Académico: Abril 2026 - Agosto 2026



Primero recordemos brevemente que la POO se trata de un paradigma de programación, en otras palabras, es una forma/método de programar que tiene como premisa base representar/modelar el mundo real, ya sea persona (tu, tus padres, etc) o cosa (carro, casa, teléfono, etc.).

¿Por qué hablar de Python y Java?

Ambos lenguajes son de alto nivel (ceranos al idioma inglés) y ambos permiten usar programación orientada a objetos, más hay diferencias y particularidades que se tratarán de explicar en el presente informe de investigación solicitado por el docente.

Clases

Recordando, una clase es como nuestro “molde” o “plano” que crea objetos, veamos como funciona y se declara en ambos lenguajes de programación:

Python:

En Python es muy sencillo, escribimos la palabra reservada “class” para dar inicio a una **nueva clase** seguida de una palabra que será el nombre, por ejemplo: `class Dog` (una clase perro).

Una vez ya establecimos la nueva clase debemos darle **atributos**, siguiendo el ejemplo de “Dog” tendríamos “species” nombre del atributo igual a “Canine” el valor que se guarda en dicho atributo, y concretamente se vería de la siguiente forma en Python: `species = “Canine”` (*Nota: a diferencia de Java aquí no hace falta definir el tipo de variable*)

Sigue el **constructor**, que es un método dedicado para poder setear los valores de los atributos, igual que con los atributos se hace diferente a Java:

```
def __init__(self, name, age);  
    (instanciamos los atributos)  
  
    self.name = name  
  
    self.age = age
```

Si nos fijamos tenemos en el constructor ciertas palabras, `__init__()` es el constructor en sí y `self` es una palabra reservada que se refiere al objeto actual así permitiendo que cada objeto setee y acceda a los valores guardados en sus atributos.

Ejemplo práctico en código Python:

```
class Dog:
    species = "Canine" # Class attribute

    def __init__(self, name, age):
        self.name = name # Instance attribute
        self.age = age # Instance attribute
```

Ya que se habló del constructor, los **métodos** los cuales tanto en java como python son los que definen el comportamiento de un objeto, siempre llevan self en los parámetros para acceder a los datos del objeto, por ejemplo:

```
def saludar(self):
    print(f"Hola, soy {self.name}")
```

El método que permite imprimir el objeto y ver sus datos en lugar de un código de memoria es `__str__`, el equivalente a `toString()` en java, por ejemplo:

```
def __str__(self):
    return f"Estudiante: {self.nombre}"
```

Tengamos en cuenta que la forma de mostrar los **niveles de acceso o encapsulamiento** en python, a diferencia de java que se hace con palabras reservadas, se hace con guiones: público (Sin guiones), `_protegido` (Un solo guión), `__privado` (Dos guiones)

Java:

En Java también es sencillo con sus diferencias, escribimos "public class" para dar inicio a una **nueva clase** seguida de una palabra que será el nombre, usaremos el mismo ejemplo anterior: public class Dog (una clase perro).

Una vez ya establecimos la nueva clase debemos darle **atributos** pero aquí hay una gran diferencia con Python, java nos exige especificar el **nivel de acceso al atributo o encapsulamiento** con palabras reservadas, recordemos java tiene tres niveles privado, protegido y público (en sintaxis Java private, protected, public respectivamente) como especificar el tipo de dato (String, int, double,... etc.); continuando el ejemplo de "Dog" tendríamos "private" el nivel de acceso al atributo "String" el tipo de

dato del atributo seguido de “species” nombre del atributo igual a “Canine” el valor que se guarda en dicho atributo, y concretamente se vería de la siguiente forma:

```
private String species = “Canine”;
```

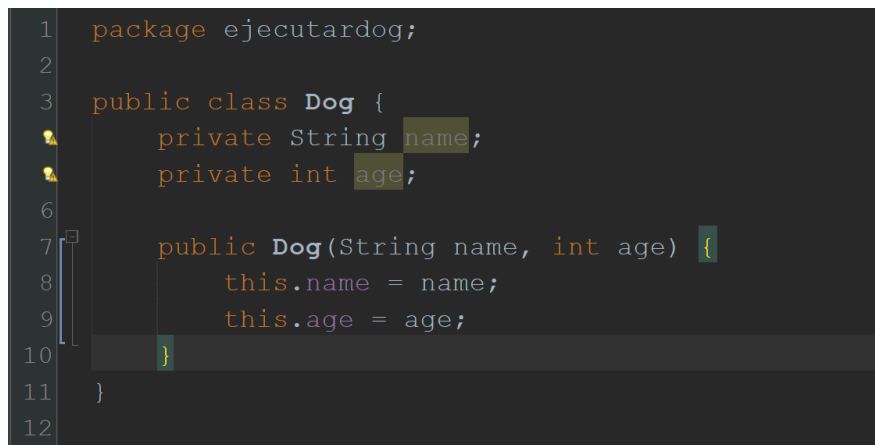
(Nota: no olvidar el “;” al final de la línea en java es Obligatorio)

Sigue el **constructor**, que es un método dedicado para poder setear los valores de los atributos, igual que con los atributos se hace diferente a Python:

```
public Dog (String name, int age){  
    (instanciamos los atributos)  
    this.name = name;  
    this.age = age;  
}
```

Si nos fijamos tenemos en el constructor ciertas palabras, publicDog(){} es el constructor en sí, los valores dentro del paréntesis son los que queremos que nos ingrese el usuario y “this.” es una palabra reservada que se refiere al objeto actual así permitiendo que cada objeto setee y acceda a los valores guardados en sus atributos no olvidemos otra vez los “;” al final de las líneas.

Ejemplo práctico en código Java:



```
1 package ejecutardog;  
2  
3 public class Dog {  
4     private String name;  
5     private int age;  
6  
7     public Dog(String name, int age) {  
8         this.name = name;  
9         this.age = age;  
10    }  
11 }  
12
```

A diferencia de python, en java para definir los **métodos** se debe insertar su respectivo modificador de acceso y un tipo de retorno, por ejemplo:

```
public void saludar (){  
    system.out.println(“Hola, soy ” + name);
```

```
}
```

En java, en cambio, el método `toString()`, se necesita especificar el `@Override` para que el compilador sepa que se está modificando el método original de la clase `Object` de java, e igual que todo lo demás en java, usa los modificadores de acceso y el tipo de retorno correspondiente, por ejemplo:

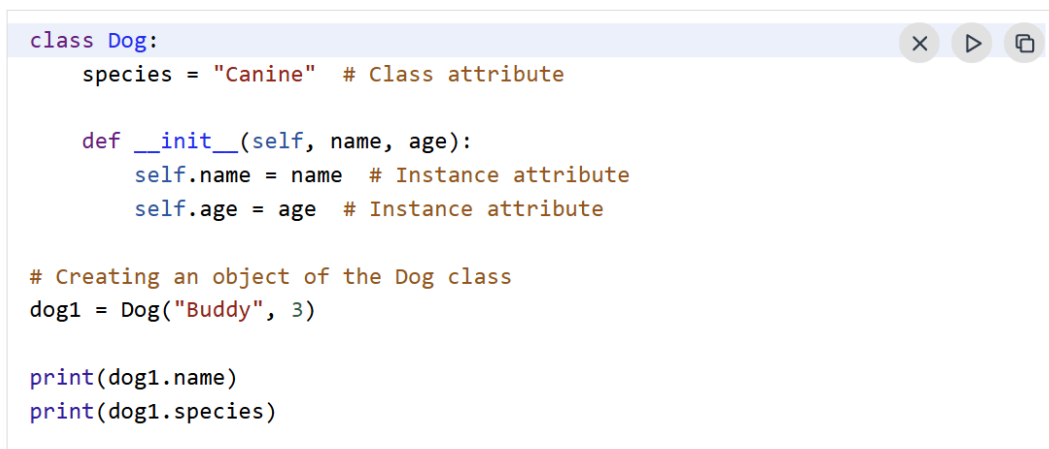
```
@Override  
  
public String toString(){  
    return "Estudiante: " + nombre;  
}
```

Ejecutar e instanciar la clase

Necesitamos crear un objeto para poder ejecutar e instanciar nuestra clase `Dog`, veamos cómo se hace en ambos lenguajes de programación.

Python:

Para **instanciar** nuestro objeto, escribimos una palabra como “dog1” de la imagen para que sea el nombre del objeto seguido de igual a “Dog” lo que significa que mi dog1 es de tipo Dog, continuamos con el paréntesis que es el constructor y los valores dentro suyo son los valores que queremos setear en los atributos de la clase Dog.



```
class Dog:  
    species = "Canine" # Class attribute  
  
    def __init__(self, name, age):  
        self.name = name # Instance attribute  
        self.age = age # Instance attribute  
  
# Creating an object of the Dog class  
dog1 = Dog("Buddy", 3)  
  
print(dog1.name)  
print(dog1.species)
```

Y para **invocar** algún método del objeto instanciado ponemos el nombre del objeto seguido por un “.” y la llamada al método correspondiente, por ejemplo:

```
dog1.saludar()
```

Java:

Como se puede ver en la imagen, en java la sintaxis es más larga y compleja, pues para **instanciar** el objeto primero debemos decirle qué clase (plano) usaremos para darle vida, seguido del nombre del objeto a su vez seguido por “=” seguido de la palabra reservada “new” luego ponemos otra vez el nombre de la clase y finalmente el constructor al cual en este ejemplo se le pasan dos variables (name y age) que contienen los valores que se desea setear en los atributos del objeto.

```

1  package ejecutardog;
2
3
4  public class EjecutarDog {
5      public static void main(String[] args) {
6          //Creo estas varibales para poder pasarle
7          //los valores al constructor del objeto
8          String name = " Buddy";
9          int age = 3;
10
11         //Creo mi objeto dog1
12         Dog dog1 = new Dog(name, age);
13
14         System.out.println("Nombre " + dog1.getName());
15     }
16

```

Y para **invocar** algun metodo del objeto instanciado ponemos el nombre del objeto seguido por un “.” y la llamada al método correspondiente pero se le añade un “;” ya que estamos en java, por ejemplo:

```
dog1.saludar();
```

Ejemplo práctico en google colab

[Codigo Python ejemplo practico - Colab](#)

Tabla comparativa

Aspecto	Java	Python	Análisis
Sintaxis y Escritura	Usa llaves y punto y coma, requiere bastante código para estructurar las clases y métodos.	Usa indentación o sangría, es más minimalista.	Python prioriza el desarrollo rápido mientras que java prioriza la estructura rígida del código para mayor orden.

Declaración de variables	Tiene tipado estático, es decir, se debe especificar si es String, int, etc.	Tiene tipado dinámico, no se declara el tipo, Python lo deduce.	Java detecta errores de tipo antes de ejecutar, mientras que Python es más flexible pero requiere de más pruebas.
Constructor	Tiene el mismo nombre que la clase y no usa parámetros obligatorios de instanciamiento.	Se usa el método especial <code>__init__</code> y requiere obligatoriamente el parámetro <code>self</code> .	<code>self</code> en python hace explícito que el atributo pertenece al objeto donde se encuentra el constructor.
Encapsulamiento	Usa palabras reservadas: <code>public</code> , <code>private</code> , <code>protected</code> .	Usa guiones bajos, sin guion, un guion o dos guiones.	En java la privacidad es forzada por el compilador, en python es más como una sugerencia.
Representación del objeto	Se usa el método <code>toString()</code>	Se usa el método <code>__str__</code>	Ambos sirven para depurar y mostrar los datos del objeto de forma humanamente legible.